LAB04A

Revisar e Reforçar a Análise Exploratória de Dados

UFRGS DELET Análise de Dados e Engenharia de Características ©Dalbiroot, Alexandre

Objetivos

Revisitar e reforçar o estudo experimental sobre Análise Exploratória de Dados (EDA). Revisar conceitos básicos sobre *pipeline* e engenharia de características para nivelar os conhecimentos de todos com relação a área de Inteligência Computacional. Este laboratório apresenta de forma bem detalhada como lidar com os problemas mais comuns encontrados em conjuntos de dados.

Conceitos Teóricos Adicionais

Revisar conceitos teóricos e experimentais apresentados nas aulas do Curso de Especialização Inteligência Artificial para Engenheiros. Apenas para registrar a biblioteca *Scikit-Learn* foi criada em 2007 por David Cournapeau como parte do projeto *Google Summer of Code* e é basicamente uma biblioteca Python de código aberto desenvolvida para facilitar o processo de construção de algoritmos estatísticos e de *Machine Learning* (ML) – na atualidade é constantemente melhorada por colaboradores. Caso tenha interesse em pesquisar mais sobre esta biblioteca sugiro acessar: https://scikit-learn.org/ e para conhecer algumas empresas que a usam acessar: https://scikit-learn.org/stable/testimonials/testimonials.html. É possível listar uma grande quantidade de vantagens no uso desta biblioteca, mas gostaria de ressaltar as principais desvantagens: (a) devido a sua facilidade de uso a mesma é pouco flexível, ou seja, o usuário não tem muita liberdade no ajuste de parâmetros ou na construção da arquitetura do modelo – isso pode acarretar sérios problemas para quem trabalha com problemas mais complexos e (b) esta biblioteca **não é considerada, por muitos, a melhor opção para quem trabalha com aprendizagem profunda**. Além disso, é importante ressaltar que a escolha do

algoritmo de *Machine Learning* (ML) adequado nem sempre é uma tarefa simples, pois muitas vezes pode não existir apenas uma solução para um problema envolvendo dados e/ou não existir um algoritmo que se adapta a todos os problemas que envolvam dados. Portanto, a escolha do(s) melhor(es) algoritmo(s) para um dado problema envolvendo dados é muitas vezes o ponto de virada em comparação as soluções medíocres. Este tipo de decisão vai necessitar um trabalho árduo da parte de vocês e muito estudo no decorrer das disciplinas deste curso, assim como, um avançado conhecimento da área ou das áreas de interesse (aplicação ou aplicações) – tarefa inerente a qualquer pessoa que trabalha nesta área de ML (*Machine Learning*) e DL (*Deep Learning*). Para esta árdua e essencial escolha sugiro seguir as seguintes etapas preliminares:

(1) não despreze os dados, ou seja, entenda os dados ("gosto de brincar com esta frase: "namore os dados"): os dados são a chave para poder desenvolver qualquer boa solução de ML ou DL, sendo assim, o primeiro passo é compreender os dados para ter a capacidade de filtrar qualquer algoritmo que seja incapaz de processar tais dados. Observar que o número de características (ou instâncias) de um conjunto de dados depende do tipo de problema, da quantidade de saídas, da quantidade de medidas, da qualidade das medidas, da significância das características envolvidas, se as características envolvidas são dependentes ou independente entre si e assim por diante; (2) após o primeiro item é importante categorizar o tipo de problema com base nos dados que possui. Sendo assim você deve analisar seus dados de entrada para determinar, por exemplo, se o conjunto de dados é estruturado (tipicamente dados organizados em tabelas) ou não estruturado (por exemplo as imagens). Além disso, é necessário avaliar se todo o conjunto de dados contém um rótulo (ou alvo ou saída desejada), ou se parte menor dos dados contém um rótulo e a parte maior não possui rótulos, ou se todos os dados não possuem rótulos e etc. Assim será possível determinar se a base de dados é supervisionada, semi-supervisionada ou não supervisionada e com isso selecionar a família de algoritmos adequada, ou seja, se os dados possuem características (entradas) e rótulos (saída desejada) o algoritmo selecionado será um algoritmo de aprendizagem supervisionada e (3) na sequência você deve escolher um conjunto de algoritmos considerando-se as etapas (1) e a (2). Assim será possível avaliar qual o melhor algoritmo do conjunto de algoritmos

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Bolbinot, Alexandre

selecionados – outros assuntos como alguns dos métodos para "alavancar o aluno mais fraco" são assuntos tratados ao longo de outras disciplinas do curso. **Mas é importante observar que é uma boa prática trabalhar e avaliar mais de um algoritmo nas aplicações de interesse de vocês**.

Existem alguns algoritmos de detecção de *outliers* bem interessantes na literatura e extremamente úteis para engenheiros de características que tipicamente trabalham com conjuntos de dados. De forma resumida: ***outlier*** é simplesmente um dado que é muito diferente de outros dados de um determinado conjunto de dados. Por exemplo, se a renda média de um determinado bairro varia entre R$5.000,00 e R$20.000,00 por família, uma família que mora neste bairro com uma renda entre R$750,00 e R$1.200,00 pode ser considerada como um dado atípico. Uma área de grande interesse do nosso laboratório é a área denominada de **Detecção de Anomalias** que basicamente está relacionada ao processo de encontrar *outliers* em um conjunto de dados. Evidentemente esta área recente tem diversas aplicações em diferentes setores, como por exemplo, em bancos, na área de finanças de forma geral, em sistemas de manufatura, em seguradoras de seguros, na indústria de forma geral (detecção de comportamentos anormais de máquinas e equipamentos em termos de manutenção preventiva e previsão de falhas), entre diversas outras opções. O algoritmo DBSCAN (já estudado em aula) é um dos algoritmos que pode ser usado para detecção de anomalias, e uma vez que já foi abordado, ele não será revisitado nesta seção. Outro algoritmo bem interessante para este tipo de aplicação é o conhecido **Floresta Isolada**. Basicamente é um algoritmo de ML não supervisionado que pode ser usado para detecção de anomalias que funciona com base no princípio de isolar anomalias e é baseado na Árvore de Decisão (assunto abordado de forma mais aprofundado na disciplina de Modelos Clássicos de Aprendizagem de Máquina). O próprio nome indica o que este algoritmo realiza, ou seja, isola características selecionando aleatoriamente uma característica de um conjunto de dados e selecionando aleatoriamente um valor dividido entre os valores mínimo e máximo dessa característica. Portanto, este algoritmo seleciona um valor de divisão aleatório dentro do intervalo dessa característica. Na sequência, se o valor escolhido mantiver o dado acima, altera o valor mínimo do intervalo de característica para esse valor, caso contrário, se o valor escolhido permanecer um dado abaixo, altera o valor

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Balbinot, Alexandre

máximo do intervalo de característica para esse valor. E repete essas duas últimas etapas até que os dados estejam isolados. O número de vezes que essas etapas são repetidas é denominada de "número de isolamento". Quanto mais baixo o número de isolamento, mas anômalo este dado é, pois, o particionamento aleatório da característica resultará em árvore com caminho mais curto para dados anômalos (menor profundidade). Portanto, os dados anômalos requerem um menor número de partições aleatórias para ser detectado como anomalia. Em Python este parâmetro é chamado "n_estimators". Para determinar o escore de anomalia em uma floresta de isolamento pode-se usar esta equação:

$$s(x,n) = 2^{-\frac{E(h(x))}{c(n)}}$$

onde $c(n) = 2H(n-1) - \left(\frac{2(n-1)}{n}\right)$, sendo n o número de dados, $c(n)$ é uma métrica que normaliza o escore entre 0 e 1 para facilitar a interpretação. $E\big(h(x)\big)$ é a média dos comprimentos do caminho da floresta de isolamento. O comprimento do percurso indica se um dado é normal ou atípico. H é um número harmônico e pode ser estimado por:

$$H(i) = ln(i) + \gamma$$

onde $\gamma = 0{,}5772156649$ (conhecido como a constante de Euler-Mascheroni). Outro algoritmo de ML não supervisionado bastante útil para identificar outliers é o LOF (*Local Outlier Factor*) que basicamente identifica vizinhos locais em vez de trabalhar com toda a distribuição dos dados. Assim como DBSCAN, o algoritmo LOF é um algoritmo baseado em densidade que utiliza a busca do vizinho mais próximo para identificar dados anômalos, sendo assim, o LOF identifica dados discrepantes (anômalos ou os *outliers*) em relação a um cluster local de dados. Neste caso os vizinhos de um determinado dado são identificados e comparados com a densidade de dados vizinhos. Os principais passos deste algoritmo são:

1. calcular a distância entre P e todos os dados usando alguma métrica de distância, por exemplo, Euclidiana ou Manhattan;
2. encontrar o k vizinho mais próximo: por exemplo, se k = 3 encontre a distância do terceiro vizinho mais próximo;
3. encontrar os k dados mais próximos;

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Balbinot, Alexandre

4. encontrar a densidade de acessibilidade local usando a seguinte equação:

$$lrd_k(O) = \frac{\|N_k(O)\|}{\sum_{O' \in N_k(O)} distância_k(O' \leftarrow O)}$$

onde distância pode ser calculada por (sendo $N_k(O)$ *o número de vizinhos*):

$$distância_k(O' \leftarrow O) = máx\{dist_k(O), dist(O, O')\}$$

5. o último passo é calcular o fator outlier local pela equação:

$$LOF_k(O) = \frac{\sum_{O' \in N_k(O)} \frac{lrd_k(O')}{lrd_k(O)}}{\|N_k(O)\|}$$

Datas Importantes

Aula(s) presencial(is) em ambiente de laboratório destinada a esta atividade: 26/03.

Data da entrega deste relatório: não será entregue relatório desta atividade.

Procedimento de Avaliação deste Laboratório:

Não será apresentada nenhuma tarefa para avaliação relacionada a este laboratório.

Procedimentos

Digitar, executar e avaliar os seguintes códigos:

1.1 Materiais e Ferramentas Computacionais

- 01 Computador adequado
- 01Ambiente adequado para programar em Python com a instalação das bibliotecas necessárias as atividades (Numpy, Pandas, Matplotlib e etc)
- Acesso ao Moodle© e se necessário à nossa comunidade no MCOnf©

1.2. Desenvolver:

i. *Tarefa no Laboratório*: vamos iniciar a Análise Exploratória de Dados (EDA) com um conjunto de dados bem simples, mas extremamente didático para percebermos problemas comuns em dados. Para isso ***baixar o arquivo Lab04A_Parte01_IAENG.html*** disponibilizado no Moodle©. Abrir o ambiente de programação Python, utilizar o código disponibilizado, executar e analisar o mesmo.

ii. *Tarefa no Laboratório*: vamos continuar exercitando algumas técnicas de EDA.

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Bulbinot, Alexandre

Inicialmente carregar um conjunto de dados e avaliar sua matriz experimental para revisar e nivelar a nossa base de conhecimentos para esta disciplina. Para isso ***baixar os arquivos Lab04A_Parte02_IAENG.html e precoscasas02.csv*** disponibilizados no Moodle©. Nesta tarefa vamos lidar com dados problemáticos, iremos transformar alguma característica no formato de texto em dados numéricos, reforçar o conceito de redimensionamento dos dados (uma das etapas de pré-processamento e da Engenharia de Características), reforçar conceitos essenciais sobre *outliers* e sua identificação. Relembrando: *outliers* podem ser entendidos como valores extremos e extraordinariamente altos ou baixos que ocorrem em um conjunto de dados. *Outliers* podem ocorrer devido a vários fatores, como por exemplo, um determinado conjunto de dados possui valores extremos genuínos, por erros de medição, erros de entrada de dados e erros de processamento de dados. É muito importante identificar e lidar com esses valores discrepantes, porque eles podem levar a conclusões erradas e distorcer qualquer análise. Nesta tarefa vamos utilizar algumas métricas da estatística e o gráfico boxplot para auxiliar nesta identificação. Além disso, vamos reforçar conceitos essenciais sobre *outliers* univariados e sua identificação. *Outliers* univariados (ou valores extremos univariados) são valores muito grandes ou pequenos que ocorrem em uma única variável de um determinado conjunto de dados. É necessário identificar este tipo de *outlier* e lidar com ele antes da realização de qualquer análise ou modelagem. Existem dois métodos principais para identificar *outliers* univariados: (a) medidas estatísticas: podemos utilizar algumas métricas ou métodos estatísticos (intervalo interquartil (IQR), o escore z e a medida da assimetria (muito conhecida pelo próprio termo em inglês *skewness*)) e (b) pela visualização dos dados com várias opções interessantes de gráficos (histogramas, boxplot, gráficos de violino e etc). Assim como, reforçar conceitos essenciais sobre *outliers* bivariados e sua identificação. Os valores extremos bivariados são geralmente valores grandes ou pequenos que ocorrem em duas variáveis simultaneamente. Em termos simples, esses valores diferem de outras observações quando examinamos as duas variáveis juntas. Individualmente, os valores em cada variável podem ou não ser outliers; No entanto, coletivamente, eles são outliers.

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Bolbinot, Alexandre

Para detectar outliers bivariados, normalmente precisamos verificar o relacionamento entre as duas variáveis. Um procedimento tradicional é primeiro visualizar se existe uma relação entre variáveis usando um gráfico de dispersão. Às vezes, podemos estar interessados em identificar valores extremos em uma variável numérica entre categorias de uma variável categórica ou valores discretos; nesse caso, um gráfico do tipo boxplot é mais útil, pois podemos identificar facilmente outliers contextuais, que geralmente são observações consideradas anômalas em um dado contexto específico. O outlier contextual se desvia significativamente do restante dos dados em um contexto específico. Por exemplo, ao analisar os preços das casas, podemos analisar atributos como a localização, o número de quartos junto com ele. É improvável que os extremos contextuais apareçam como discrepantes quando apenas análises univariadas são realizadas nos preços das casas. Eles são frequentemente identificados quando consideramos os preços da casa dentro de um contexto específico. As características ou atributos das casas, como localização ou número de salas, são exemplos desse contexto específico. Novamente, é importante entender o contexto e o objetivo da análise antes de escolher um método apropriado para lidar com *outliers* bivariados. Adicionalmente vamos também reforçar conceitos essenciais sobre *outliers* multivariados e sua identificação. *Outliers* multivariados são valores extremos que ocorrem em três ou mais variáveis simultaneamente. Podemos identificar outliers multivariados usando, por exemplo, as seguintes técnicas: (a) distância Mahalanobis: mede a distância de cada observação do centro da distribuição de dados (centróide). Em termos simples, a distância Mahalanobis ajuda a quantificar o quão longe, cada dado é do valor médio (centróide ou localização central das variáveis no conjunto de dados); (b) análise por *cluster*: ajuda a agrupar observações semelhantes e a encontrar *outliers* multivariados. O algoritmo DBSCAN é um algoritmo popular que pode ser usado para esta detecção. Basicamente irá identificar grupos de dados que estão próximos e irá considerar dados que não fazem parte de nenhum *cluster* como *outliers* (observar que este algoritmo também considera os dados em pequenos *clusters* como *outliers*) e (c) pela visualização gráfica. Finalmente vamos abordar um método simples para remoção de *outliers* que

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Bulbinot, Alexandre

basicamente envolve a remoção completa dos valores discrepantes antes de analisar nosso conjunto de dados – procedimento conhecido como corte. A grande desvantagem este procedimento é que podemos perder alguns insights úteis especialmente se os outliers forem legítimos, ou seja, eles podem revelar informações importantes que serão negligenciadas se forem removidas. Porém, o corte pode ser útil quando a distribuição dos dados é muito importante e quando temos um número pequeno de outliers. Na sequência vamos substituir os *outliers*. Uma possível abordagem é substituir os valores extremos por uma valor predeterminado. Assim como a remoção de *outliers*, isso precisa ser feito com bastante cuidado, pois pode introduzir viés em nosso conjunto de dados. Os principais métodos empregados são: (a) uso de métricas estatísticas: envolve a substituição de *outliers* pela média, mediana ou percentis do conjunto de dados; (b) interpolação: envolve estimar o valor de um *outlier* usando os dados vizinhos do *outlier* e (c) métodos baseados em modelo: envolvem o uso de um modelo inteligente para prever o valor de reposição para os *outliers*. Porém, é importante observar que os métodos anteriores afetarão a forma e as características da distribuição do conjunto de dados e não são apropriadas em cenários em que a distribuição dos dados é importante. Vamos também detalhar os valores ausentes utilizando algumas funções do Pandas e do Missingno e na sequência vamos deletar os valores ausentes. Basicamente é a abordagem mais simples e tipicamente empregada nas ausências do tipo MCAR. Podemos remover os valores de uma lista, ou seja, executar a remoção de todos os dados que contêm um ou mais valores ausentes (método também conhecido como análise completa, pois apenas casos completos são analisados). Também podemos executar uma remoção em pares onde apenas os dados disponíveis para cada variável são usados para análise. Neste tipo de situação o valor ausente é ignorado/excluído, enquanto para variáveis sem o valor ausente, o valor presente é usado para análise. A exclusão em pares também é conhecida como análise de caso disponível. A exclusão da lista acarreta à perda de dados, enquanto a exclusão pareada mantêm dados. Muitas vezes, ambos os métodos podem introduzir viés no conjunto de dados. Outra abordagem que pode ser usada para remover valores ausentes é a remoção de colunas inteiras onde existem

UFRGS DELET, Análise de Dados – Engenharia de Características, ©Balbinot, Alexandre

valores ausentes – normalmente se adota este exclusão quando a coluna inteira é preenchida com valores ausentes ou a maioria dos valores da coluna estão ausentes. Finalmente vamos substituir os valores ausentes. Evidentemente a remoção de valores ausentes é uma abordagem simples e rápida para lidar com valores ausentes, porém só é uma metodologia eficaz quando os valores ausentes são poucos no conjunto de dados. Portanto, substituir os valores ausentes é uma abordagem melhor quando existem muitos valores ausentes em um conjunto de dados principalmente se envolverem variáveis críticas **E** esses valores estão ausentes aleatoriamente. Esta abordagem de substituir os valores ausente é conhecida como **imputação** e as principais abordagens são: (a) uso de métricas estatísticas: envolve o uso de estatísticas resumidas como média, mediana, percentis e assim por diante; (b) preenchimento para frente ou pelo último valor: em dados sequenciais podemos usar o último valor antes do valor ausente ou o próximo valor após o valor ausente. Método é apropriado para quem trabalha com séries temporais, onde os valores ausentes provavelmente dependem do tempo e (c) métodos baseado em modelos inteligentes: envolve o uso de modelos de aprendizagem de máquina, como por exemplo, a regressão linear ou o algoritmo de *cluster* vizinhos mais próximos (k-NN). No final deste extenso laboratório vamos substituir os valores ausentes usando a imputação pelo uso de modelos inteligentes. Um método muito popular é usar a imputação pelo uso do k-NN. Basicamente o modelo deve identificar o *cluster* que engloba os valores ausentes e utilizar a média desse *cluster* para substituir os valores ausentes. Existem vários outros modelos inteligentes que podem ser usados, como regressão linear, árvores de decisão, entre outros. Nesta tarefa vamos utilizar o k-NN e deixo como sugestão testarem outros modelos com este mesmo objetivo.

iii. *Tarefa no Laboratório*: vamos avaliar agora uma base de dados relacionada com a COVID19 contendo vários dados ausentes. Este conjunto de dados envolve amostras que representam pacientes que procuraram ajuda médica apresentando sintomas de uma possível doença de ordem respiratória. As características representam informações sobre os pacientes e sintomas que estão apresentando. O rótulo ou a variável de saída (ou de resposta) tem duas opções: (a) COVID-19 (doença causada pelo

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Bolbinot, Alexandre

SARS-CoV-2) e (b) H1N1 (subtipo de influenza). Para isso ***baixar os arquivos Lab04A_Parte03_IAENG.html e covid.csv*** disponibilizados no Moodle©. Nesta atividade vamos trabalhar com a técnica de imputação.

iv. *Tarefa no Laboratório*: vamos continuar trabalhando com a mesma base de dados da *Tarefa iii*. Observar que acabei trabalhando este código na sequência do anterior, porém sugiro depois com mais tempo vocês retirem as partes desnecessárias, ou seja, trechos desnecessários do código que foram usados na tarefa anterior. Nesta tarefa apresentarei as principais opções de transformação dos dados (características especialmente). Apenas para reforçar a compreensão desta tarefa é essencial apresentar a equação da chamada Transformação Box-Cox, pois infelizmente não é tão comum de ser usada, mas é muito interessante para modelos onde existe uma forte dependência que os dados sigam uma distribuição normal. A transformação Box-Cox é representada pela seguinte equação (observar que essa equação funciona apenas com dados estritamente positivos):

$$x_i^{(\lambda)} = \begin{cases} \dfrac{x_i^{\lambda} - 1}{\lambda}, se\ \lambda \neq 0 \\ lnx(x_i), se\ \lambda = 0 \end{cases}$$

Para análise desta atividade ***baixar os arquivos Lab04A_Parte04_IAENG.html e covid.csv***. Abrir o ambiente de programação Python, utilizar o código disponibilizado no arquivo e analisar o mesmo.

v. *Tarefa no Laboratório*: vamos implementar o algoritmo floresta isolada (também chamado por muitos de floresta de isolamente) usando a biblioteca scikit-learn para organizar os valores discrepantes de uma base de dados. Esta base apresenta a renda de engenheiros versus o conjunto de dados de gastos habituais (hábitos de consumo). Para isso baixar no Moodle os arquivos ***Dados_Engenheiro.csv*** e ***Lab04A_Parte05_IAENG.html***. Abrir o ambiente desejado de programação Python e digitar o código disponibilizado (realizar uma análise passo a passo e, se necessário, consultar o *help* das ferramentas utilizadas).

vi. *Tarefa no Laboratório*: vamos implementar o LOF usando a mesma base de dados da *Tarefa v*. Para isso baixar no Moodle os arquivos ***Dados_Engenheiro.csv*** e

UFRGS, DELET, Análise de Dados e Engenharia de Características, ©Bolbinot, Alexandre

Lab04A_Parte06_IAENG.html. Abrir o ambiente desejado de programação Python e digitar o código disponibilizado (realizar uma análise passo a passo e, se necessário, consultar o *help* das ferramentas utilizadas).

vii. *Tarefa no Laboratório*: métodos de agrupamento de partição, como k-means, e métodos de agrupamento hierárquico, como clustering aglomerativo, são bons para descobrir clusters esféricos ou convexos. Esses algoritmos são mais sensíveis a ruídos ou *outliers*, sendo assim, funcionam bem para clusters bem separados. Por outro lado, o algoritmo DBSCAN é baseado na ideia de que cada dado de um grupo ou cluster tem um número mínimo de dados em um determinado raio. O principal objetivo do DBSCAN é descobrir a região densa que pode ser calculada usando o número mínimo de dados (NúmMín) e um determinado raio (epsilon). O algoritmo DBSCAN tem a capacidade de gerar formas aleatórias de clusters e "lidar melhor" com o ruído de um conjunto de dados. Também, não há necessidade de determinar o número de clusters, como por exemplo, no k-means, ou seja, o algoritmo DBSCAN identifica automaticamente o número de clusters de um conjunto de dados. Nesta atividade vamos implementar o algoritmo DBSCAN usando a biblioteca scikit-learn para avaliar o impacto do tamanho do raio de vizinhança. Além disso, vamos implementar o algoritmo DBSCAN braçalmente usando um conjunto de dados bidimensional gerado. Para exercitar esta tarefa **baixar no Moodle o arquivo *Lab04A_Parte07_IAENG.html***. Abrir o ambiente desejado de programação Python e digitar o código disponibilizado (realizar uma análise passo a passo e, se necessário, consultar o *help* das ferramentas utilizadas).

viii. *Tarefa no Laboratório*: vamos implementar agora o algoritmo DBSCAN para avaliar o Impacto do Limite do Mínimo de Pontos no correspondente agrupamento. Para exercitar esta tarefa **baixar no Moodle o arquivo *Lab04A_Parte08_IAENG.html***. Abrir o ambiente desejado de programação Python e digitar o código disponibilizado (realizar uma análise passo a passo e, se necessário, consultar o *help* das ferramentas utilizadas).

ix. *Tarefa no Laboratório*: vamos comparar o algoritmo DBSCAN com o k-means e o Agrupamento Hierárquico. Para isso baixar os seguintes arquivos no Moodle: ***Lab04A_Parte09_IAENG.html*** e **vinho.csv** do MConf. Basicamente este conjunto de dados apresenta dados químicos de três tipos diferentes de vinho com várias

UFRGS DELET, Análise de Dados e Engenharia de Características, ©Bulbinet, Alexandre

características relacionadas a qualidade do vinho, como por exemplo, álcool, ácido málico; cinzas; alcalinidade das cinzas, entre outras. Abrir o ambiente de programação Python, utilizar o código disponibilizado no arquivo e analisar o mesmo.

UFRGS, DELET, Análise de Dados e Engenharia de Características, ©Balbinot, Alexandre